

MERN E-commerce

Complete Documentation

A full-stack online store: storefront, admin panel, reviews, wishlist, search, image uploads and Stripe payments.

Generated 2026-05-25

React 18 · Vite · Redux Toolkit · Express · Mongoose · Stripe

MERN E-commerce — Complete Documentation

A full-stack e-commerce platform with a customer storefront, JWT-protected admin panel, image uploads, reviews & ratings, advanced search, wishlist, and Stripe Checkout payments.

Table of contents

1. Overview
 2. Architecture
 3. Tech stack
 4. Project structure
 5. Prerequisites (Windows)
 6. Step-by-step installation
 7. MongoDB setup (local or Atlas)
 8. Environment variables
 9. Seeding sample data
 10. Running the app
 11. Default credentials
 12. Stripe payment setup
 13. Feature walkthrough
 14. REST API reference
 15. Backend — file-by-file reference
 16. Frontend — file-by-file reference
 17. Library reference (every dependency)
 18. Troubleshooting
 19. Cheat sheet
-

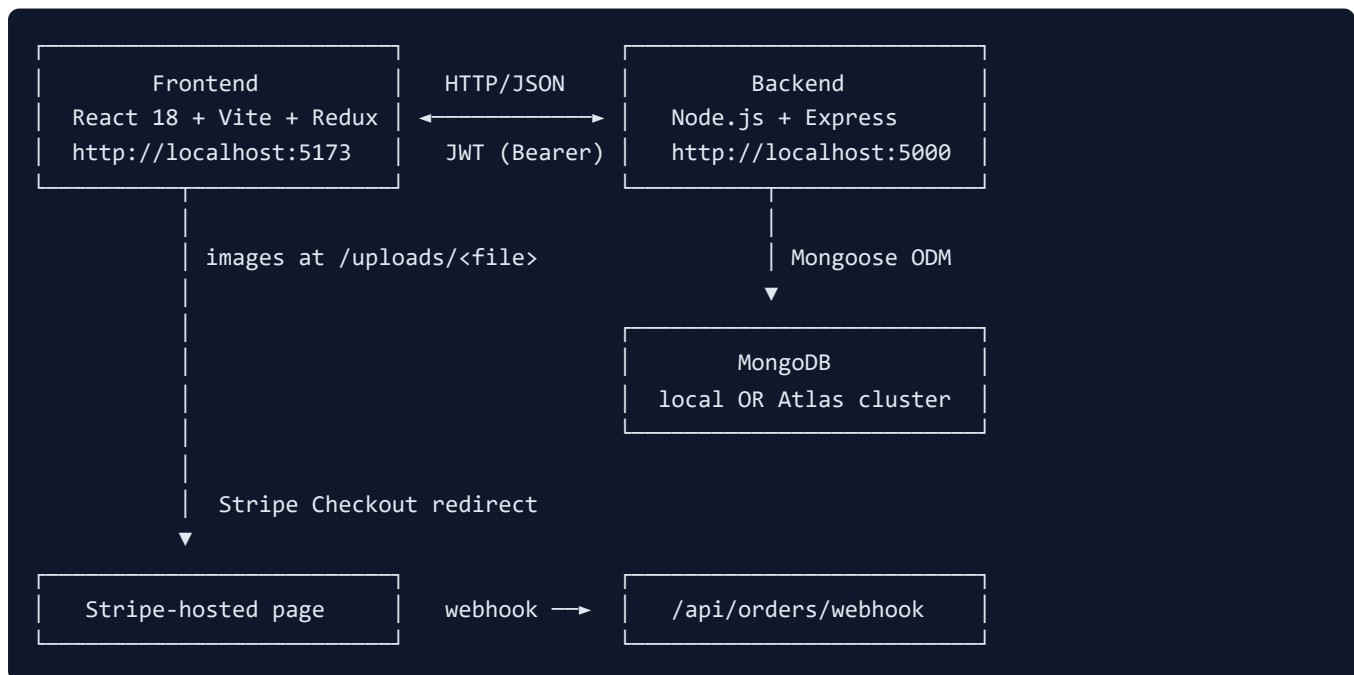
1. Overview

This project is a complete MERN (MongoDB · Express · React · Node) e-commerce application. It includes everything needed to run a small online store:

- Public storefront with product browsing, search and filters
- User registration and login (JWT)
- Cart (persisted to `localStorage`)

- Reviews and 5-star ratings per product
- Wishlist (saved per user)
- Stripe Checkout payments
- Admin panel for products, categories, orders and users
- Image upload via `multer`, or via external image URL
- Toast notifications and a top-level error boundary
- Seeder that populates the database with 30 realistic products, 6 categories, 5 users and ~75 reviews

2. Architecture



- **Frontend** → **Backend**: axios with a request interceptor that attaches `Authorization: Bearer <jwt>` from `localStorage.userInfo`.
- **Backend** → **MongoDB**: Mongoose. Connection string in `MONGO_URI`.
- **Stripe**: Checkout Session URL returned by the backend; frontend redirects there. Stripe webhooks hit `/api/orders/webhook` to mark the order as paid.

3. Tech stack

Backend

Library	Version	Purpose
express	4.19.2	HTTP server / routing
mongoose	8.5.1	MongoDB ODM
jsonwebtoken	9.0.2	JWT issuing & verification
bcryptjs	2.4.3	Password hashing
dotenv	16.4.5	Loading <code>.env</code> variables
cors	2.8.5	Permit cross-origin requests from the frontend
multer	1.4.5-lts.1	Multipart upload handling
express-async-handler	1.2.0	Wraps async controllers so thrown errors go to the error middleware
stripe	16.8.0	Stripe Checkout Sessions + webhook verification
colors	1.4.0	Coloured seeder output
nodemon (dev)	3.1.4	Hot reload during development

Frontend

Library	Version	Purpose
react / react-dom	18.3.1	UI library
react-router-dom	6.26.1	Client-side routing
@reduxjs/toolkit	2.2.7	State management (<code>createSlice</code> , <code>createAsyncThunk</code> , <code>createListenerMiddleware</code>)
react-redux	9.1.2	React bindings for Redux
axios	1.7.4	HTTP client
react-icons	5.3.0	Icon set (<code>FaShoppingCart</code> , <code>FaStar</code> , etc.)
@stripe/stripe-js	4.3.0	Available for future client-side Stripe integration (not required for the redirect flow)
vite (dev)	5.4.1	Dev server + bundler
@vitejs/plugin-react (dev)	4.3.1	Vite React plugin

4. Project structure

```
projetfinal/
├── README.md
├── docs/
│   ├── DOCUMENTATION.md    ← this file
│   └── DOCUMENTATION.pdf    ← generated
├── backend/
│   ├── server.js            Express entry point
│   ├── seeder.js            DB seeding script
│   ├── package.json         Dependencies + scripts
│   ├── .env / .env.example  Environment variables
│   ├── .gitignore
│   ├── config/
│   │   └── db.js            Mongoose connection
│   ├── middleware/
│   │   ├── authMiddleware.js protect + admin
│   │   └── errorMiddleware.js notFound + errorHandler
│   ├── models/
│   │   ├── User.js          bcrypt hook, matchPassword, wishlist
│   │   ├── Category.js
│   │   ├── Product.js       reviews subdoc, rating, numReviews
│   │   └── Order.js          items, shippingAddress, paymentResult
│   ├── controllers/
│   │   ├── userController.js
│   │   ├── categoryController.js
│   │   ├── productController.js filters, pagination, reviews
│   │   ├── orderController.js orders + Stripe session + webhook
│   │   └── wishlistController.js
│   ├── routes/
│   │   ├── userRoutes.js
│   │   ├── categoryRoutes.js
│   │   ├── productRoutes.js
│   │   ├── orderRoutes.js
│   │   ├── wishlistRoutes.js
│   │   └── uploadRoutes.js   multer upload endpoint
│   ├── data/
│   │   ├── users.js          Seed users (1 admin + 4 customers)
│   │   ├── categories.js     Seed categories (6)
│   │   └── products.js       Seed products (30 with real Unsplash URLs)
│   ├── utils/
│   │   └── generateToken.js
│   └── uploads/              Multer destination (gitignored)
├── frontend/
│   ├── index.html
│   ├── vite.config.js
│   ├── package.json
│   ├── .env
│   ├── public/
│   │   └── placeholder.svg   Image-fallback asset
│   └── src/
│       ├── main.jsx          React entry – Provider + ErrorBoundary + Router + Toast
│       └── App.jsx            Route table
```

```
|— services/api.js          axios instance + JWT interceptor
|— utils/imageUrl.js       image URL resolver + onError fallback
|— redux/
|   |— store.js            configureStore + cart persistence middleware
|   |— slices/
|       |— productSlice.js
|       |— cartSlice.js
|       |— userSlice.js
|       |— orderSlice.js
|       |— wishlistSlice.js
|— components/
|   |— Navbar.jsx
|   |— Footer.jsx
|   |— ProductCard.jsx
|   |— Rating.jsx
|   |— Loader.jsx
|   |— PrivateRoute.jsx
|   |— AdminRoute.jsx
|   |— ErrorBoundary.jsx
|   |— Toast.jsx
|— pages/
|   |— HomePage.jsx
|   |— SearchPage.jsx
|   |— ProductPage.jsx
|   |— CartPage.jsx
|   |— LoginPage.jsx
|   |— RegisterPage.jsx
|   |— ProfilePage.jsx
|   |— CheckoutPage.jsx
|   |— WishlistPage.jsx
|   |— OrderPage.jsx
|   |— admin/
|       |— DashboardPage.jsx
|       |— ProductListAdmin.jsx
|       |— ProductEditPage.jsx
|       |— CategoryListAdmin.jsx
|       |— OrderListAdmin.jsx
|       |— UserListAdmin.jsx
|— styles/
|   |— global.css
|   |— toast.css
|   |— navbar.css
|   |— product.css
|   |— cart.css
|   |— forms.css
|   |— admin.css
|   |— reviews.css
|   |— search.css
|   |— wishlist.css
|   |— checkout.css
```

5. Prerequisites (Windows)

Install once per machine:

1. **Node.js LTS 18+** — <https://nodejs.org>
2. **Git** — <https://git-scm.com/>
3. **MongoDB**, either:
 - **Local:** MongoDB Community Server from <https://www.mongodb.com/try/download/community> (accept "Install MongoDB as a Service" during setup), or
 - **Cloud:** A free Atlas cluster — <https://www.mongodb.com/cloud/atlas/register>
4. (optional) **Stripe CLI** for webhook testing — <https://stripe.com/docs/stripe-cli>
5. (optional) **VS Code** — <https://code.visualstudio.com/>

Verify Node:

```
node --version    # v18.x or higher
npm --version
```

A note about PowerShell 5.1

Windows 10/11 ship with PowerShell 5.1 which **does not support** `&&` as a command separator. Either:

- Run commands on separate lines, **or**
- Use `;` to chain (runs the next command even if the previous failed), **or**
- Use `; if ($?) { ... }` to chain only on success, **or**
- Install PowerShell 7+ (`winget install Microsoft.PowerShell`) — supports `&&`.

6. Step-by-step installation

```
# 1. Get the code
git clone <your-repo-url> projetfinal
cd projetfinal

# 2. Backend
cd backend
npm install
Copy-Item .env.example .env
notepad .env    # edit values (next section)

# 3. Frontend (new terminal or after cd ..)
cd ../frontend
npm install
# frontend already has a .env with the API URL; only edit if you change ports
```

Expected install output:

- backend: added ~155 packages
- frontend: added ~107 packages

7. MongoDB setup

Option A — Local MongoDB

After installing MongoDB Community Server with the "Install as a Service" option, MongoDB is already running on `mongodb://localhost:27017`. Confirm:

```
Get-Service MongoDB           # should be "Running"
Test-NetConnection localhost -Port 27017  # TcpTestSucceeded : True
```

Use this for `MONGO_URI` in `backend/.env`:

```
MONGO_URI=mongodb://localhost:27017/mern_ecommerce
```

Option B — MongoDB Atlas (cloud, free)

1. Create a free cluster at <https://www.mongodb.com/cloud/atlas/register>.
2. **Network Access** → **Add IP Address** → **Allow Access From Anywhere** (`0.0.0.0/0`) for development.
3. **Database Access** → **Add new database user** (username + password).
4. **Connect** → **Drivers** → **Node.js** and copy the connection string. It looks like:

```
mongodb+srv://<user>:<password>@cluster0.xxxx.mongodb.net/?retryWrites=true&w=majority
```

5. Insert your database name before the `?`:

```
mongodb+srv://<user>:<password>@cluster0.xxxx.mongodb.net/mern_ecommerce?retryWrites=true&w=majority
```

Use that as `MONGO_URI` in `backend/.env`.

8. Environment variables

backend/.env

```
MONGO_URI=mongodb://localhost:27017/mern_ecommerce
JWT_SECRET=replace_with_any_long_random_string
PORT=5000

# Stripe – only required to enable payments
STRIPE_SECRET_KEY=sk_test_...
STRIPE_PUBLISHABLE_KEY=pk_test_...
STRIPE_WEBHOOK_SECRET=whsec_...
CLIENT_URL=http://localhost:5173
```

frontend/.env

```
VITE_API_URL=http://localhost:5000/api
VITE_STRIPE_PUBLISHABLE_KEY=pk_test_...
```

`CLIENT_URL` (backend) and Vite's port must match. Vite defaults to **5173**.

9. Seeding sample data

The seeder wipes Users, Categories, Products and Orders and inserts:

- 5 users (1 admin + 4 customers)
- 6 categories
- 30 products with real Unsplash images
- ~75 reviews (2–3 per product)

```
cd backend
npm run seed          # destroy + insert
npm run seed:destroy  # delete all, do not reseed
```

Expected output:

```
Starting seed...
  cleared existing collections
  inserted 5 users
  inserted 6 categories
  inserted 30 products
  inserted 74 reviews

✅ Data seeded: 5 users, 6 categories, 30 products, 74 reviews
```

10. Running the app

Open **two** PowerShell terminals.

Terminal 1 — backend

```
cd C:\path\to\projetfinal\backend
npm run dev
```

Output:

```
Server running on port 5000
MongoDB Connected: ...
```

Terminal 2 — frontend

```
cd C:\path\to\projetfinal\frontend
npm run dev
```

Vite opens <http://localhost:5173> automatically.

11. Default credentials

Role	Email	Password
Admin	admin@shop.com	admin123
Customer	salma.bouزيد@gmail.com	password123
Customer	karim.trabelsi@outlook.com	password123
Customer	lucas.martin@gmail.com	password123
Customer	yasmine.mansouri@gmail.com	password123

12. Stripe payment setup

12.1 Get your API keys

1. Sign in to <https://dashboard.stripe.com>.
2. Make sure the dashboard is in **Test mode** (top-left toggle).
3. **Developers** → **API keys** — copy the **Publishable** and **Secret** keys (both start with `pk_test_` / `sk_test_`).

Paste them into `backend/.env` and `frontend/.env`.

12.2 Forward webhooks to localhost

The webhook tells the backend that Stripe has completed a payment.

In a **third terminal**:

```
stripe login # one-time browser auth
stripe listen --forward-to localhost:5000/api/orders/webhook
```

The CLI prints a `whsec_...` line. Copy that into `STRIPE_WEBHOOK_SECRET` in `backend/.env`, then **restart the backend**.

12.3 Try a payment

1. Sign in → add a product to the cart → **Proceed to Checkout**.
2. Fill the shipping form, choose **Stripe** as the payment method, click **Place Order & Pay**.
3. You land on Stripe's hosted Checkout. Use card number `4242 4242 4242 4242`, any future expiry, any 3-digit CVC, any 5-digit ZIP.
4. On success Stripe redirects back to `/order/:id?payment=success`. A toast appears, and the order page polls for ~20 s until the webhook marks the order paid.

Useful Stripe test cards: <https://stripe.com/docs/testing>

13. Feature walkthrough

Area	What to try
Search	Type in the navbar — it debounces 300 ms then navigates to <code>/search/<keyword></code> . Use the sidebar filters (category, price range, sort). URL stays shareable.
Pagination	At the bottom of <code>/search/...</code> use Prev / page numbers / Next.
Reviews	Sign in, open any product, write a review (1–5 stars + comment). Average updates immediately. Delete via the trash icon (admin can delete any review).
Wishlist	Click the heart on any product card. Visit <code>/wishlist</code> from the navbar. Add to cart or remove from there.
Cart	Adjust quantities, remove items. The cart survives page reloads (<code>localStorage</code>).
Checkout	Choose Stripe / PayPal / Cash on Delivery. Stripe redirects to Stripe Checkout; the other methods skip directly to the order page.
Admin	Log in as admin → top-right <code>Admin</code> badge → manage products, categories, orders (mark delivered) and users (toggle admin, delete).

14. REST API reference

Base URL: `http://localhost:5000/api`. JWT goes in the `Authorization: Bearer <token>` header.

14.1 Users / Auth

Method	Path	Auth	Body	Returns
POST	<code>/users</code>	public	<code>{name, email, password}</code>	<code>{_id, name, email, isAdmin, token}</code>
POST	<code>/users/login</code>	public	<code>{email, password}</code>	same
GET	<code>/users/profile</code>	user	—	current user
PUT	<code>/users/profile</code>	user	<code>{name?, email?, password?}</code>	updated user + new token
GET	<code>/users</code>	admin	—	array of users
GET	<code>/users/:id</code>	admin	—	single user
PUT	<code>/users/:id</code>	admin	<code>{name?, email?, isAdmin?}</code>	updated user
DELETE	<code>/users/:id</code>	admin	—	<code>{message}</code>

14.2 Wishlist

Method	Path	Auth	Body	Returns
GET	/users/wishlist	user	—	array of populated products
POST	/users/wishlist/:productId	user	—	201 {message, productId} (idempotent)
DELETE	/users/wishlist/:productId	user	—	204 No Content

14.3 Categories

Method	Path	Auth
GET	/categories	public
GET	/categories/:id	public
POST	/categories	admin
PUT	/categories/:id	admin
DELETE	/categories/:id	admin

14.4 Products

Method	Path	Auth	Notes
GET	/products	public	Query params: keyword, category, minPrice, maxPrice, sortBy (newest, price_asc, price_desc, rating, popular), page, limit. Returns {products, page, pages, total}
GET	/products/:id	public	category is populated
POST	/products	admin	create
PUT	/products/:id	admin	update
DELETE	/products/:id	admin	delete
POST	/products/:id/reviews	user	{rating, comment} (1 per user — duplicates → 400)
DELETE	/products/:id/reviews/:reviewId	owner / admin	204 No Content

14.5 Orders

Method	Path	Auth	Purpose
POST	<code>/orders</code>	user	create from cart
GET	<code>/orders/myorders</code>	user	current user's orders
GET	<code>/orders/:id</code>	owner / admin	order details
PUT	<code>/orders/:id/pay</code>	owner / admin	manually mark paid
GET	<code>/orders</code>	admin	all orders
PUT	<code>/orders/:id/deliver</code>	admin	mark delivered
POST	<code>/orders/:id/create-checkout-session</code>	owner	returns <code>{url, id}</code> — Stripe Checkout URL
POST	<code>/orders/webhook</code>	Stripe	raw body; webhook signature verified — flips <code>isPaid</code>

14.6 Uploads

Method	Path	Auth	Body	Returns
POST	<code>/upload</code>	admin	<code>multipart/form-data</code> with field <code>image</code>	<code>{message, path: "/uploads/<filename>"}</code>

Static files are then served at `http://localhost:5000/uploads/<filename>`.

15. Backend — file-by-file reference

15.1 `server.js`

The Express entry point. Key details:

- Loads `.env` via `dotenv.config()`.
- Calls `connectDB()` from `config/db.js`.
- Registers `cors()` then the **Stripe webhook with** `express.raw({ type: 'application/json' })` **BEFORE** `express.json()` — Stripe needs the raw byte body to verify the signature.
- Mounts the wishlist router **before** the catch-all `/api/users` router so requests to `/api/users/wishlist` aren't matched by the admin's `/users/:id` route.

- Serves `/uploads/<file>` statically.
- Registers `notFound` and `errorHandler` middlewares at the end.

15.2 config/db.js

Single function `connectDB()` that calls `mongoose.connect(process.env.MONGO_URI)`. On success logs `MongoDB Connected: <host>`. On failure logs the error and calls `process.exit(1)`.

15.3 middleware/authMiddleware.js

- `protect` — reads `Authorization: Bearer ...`, verifies the JWT with `JWT_SECRET`, loads the user (minus password) into `req.user`. On any failure throws via `express-async-handler`, which the global error middleware turns into a 401 JSON response.
- `admin` — synchronous guard that checks `req.user.isAdmin`, otherwise throws "Not authorized as admin" with status 401.

15.4 middleware/errorMiddleware.js

- `notFound` — generic 404 fall-through for unmatched routes.
- `errorHandler` — turns any thrown error into JSON `{message, stack}`. Hides the stack when `NODE_ENV === 'production'`.

15.5 utils/generateToken.js

`generateToken(userId)` signs `{id}` with `JWT_SECRET`, expiry 30 days.

15.6 Models

models/User.js

Fields: `name`, `email` (unique, lowercase), `password`, `isAdmin` (default false), `wishlist` (array of Product ObjectIds), `timestamps`.

- **Pre-save hook:** hashes the password with `bcrypt.hash(..., 10)` whenever the password field is new or modified.
- **Instance method:** `matchPassword(plain)` → returns a `Promise<boolean>` using `bcrypt.compare`.

Reminder: the hook only fires on `Model.create()` / `doc.save()`. Use those (not `insertMany`) when you want hashing.

models/Category.js

Fields: `name` (unique, trimmed), `description`, `timestamps`. Plain Mongoose schema.

models/Product.js

Fields: `name`, `description`, `price`, `image` (URL or `/uploads/...` path), `category` (ref Category), `countInStock`, `rating`, `numReviews`, `reviews` (array of `reviewSchema`), `timestamps`.

`reviewSchema` subdoc: `user` (ref User), `name`, `rating` 1–5, `comment`, `timestamps`. Subdocuments get auto-generated `_id` so the front-end can target them for deletion.

models/Order.js

Fields: `user`, `orderItems` (array of `orderItemSchema` with `product` ref + name/qty/price/image snapshot — `_id: false`), `shippingAddress` (subdoc with address/city/postalCode/country — `_id: false`), `paymentMethod`, `paymentResult` (subdoc with Stripe session metadata: `id`, `status`, `email_address`, `update_time`), `totalPrice`, `isPaid`, `paidAt`, `isDelivered`, `deliveredAt`, `timestamps`.

15.7 Controllers

All wrapped with `express-async-handler` so unhandled rejections / thrown errors flow to the error middleware.

controllers/userController.js

- `registerUser` — 201 with token if email is free; 400 if not.
- `loginUser` — `matchPassword` check; 401 otherwise.
- `getUserProfile` — uses `req.user._id`.
- `updateUserProfile` — name/email/password (setting password triggers the pre-save bcrypt hook).
- `getUsers` — admin, sorted newest first.
- `getUserById` — admin.
- `updateUser` — admin; can toggle `isAdmin`.
- `deleteUser` — admin. Refuses to delete the calling user (400).

controllers/categoryController.js

Classic CRUD: `getCategories`, `getCategoryById`, `createCategory` (admin), `updateCategory` (admin), `deleteCategory` (admin). `createCategory` returns 401 on duplicate name.

controllers/productController.js

- `getProducts` — accepts `keyword`, `category`, `minPrice`, `maxPrice`, `sortBy`, `page`, `limit`. Builds the filter, runs both `countDocuments` and `find` in series, returns `{products, page, pages, total}`.
- `getProductById` — populates `category`.
- `createProduct`, `updateProduct`, `deleteProduct` — admin.

- `createProductReview` — validates rating 1–5, rejects duplicate reviews from the same user, pushes the new review, then calls `recomputeRating` (avg + count) and saves.
- `deleteProductReview` — checks the caller is the owner or admin; otherwise 403. Recomputes after deletion. Returns 204.

controllers/orderController.js

- `addOrderItems` — 400 if items array is empty.
- `getMyOrders`, `getOrderById` (with owner/admin guard), `getAllOrders` (admin).
- `updateOrderToPaid` — owner/admin; sets `isPaid` and `paidAt`.
- `updateOrderToDelivered` — admin only.
- `createCheckoutSession` — admin/owner. Builds Stripe line items (price in cents), calls `stripe.checkout.sessions.create` with `success_url` `${CLIENT_URL}/order/<id>?payment=success` and `cancel_url` `?payment=cancelled`. Returns `{url, id}`.
- `stripeWebhook` — verifies the `Stripe-Signature` header with `STRIPE_WEBHOOK_SECRET`. On `checkout.session.completed` flips `isPaid`, sets `paidAt`, stores `paymentResult` (id, status, email, update_time). Mounted in `server.js` with `express.raw` because Stripe verification needs the literal byte buffer.

Stripe is **lazily** initialised — `getStripe()` only constructs the client when the secret key is present, so the app boots fine without payment configured.

controllers/wishlistController.js

- `getWishlist` — returns the user's wishlist populated with full Product docs.
- `addToWishlist` — uses `$addToSet`, so it's idempotent.
- `removeFromWishlist` — uses `$pull` and returns 204.

15.8 Routes

- `routes/userRoutes.js` — `/`, `/login`, `/profile`, then `/:id` for the admin endpoints.
- `routes/wishlistRoutes.js` — `/`, `/:productId`. Mounted at `/api/users/wishlist` in `server.js`.
- `routes/categoryRoutes.js`, `routes/productRoutes.js`, `routes/orderRoutes.js` — REST routers wiring each controller method with `protect` / `admin` guards as required.
- `routes/uploadRoutes.js` — multer disk storage that writes into `backend/uploads/`. Filenames are sanitized + timestamped. Whitelist: `jpg`, `jpeg`, `png`, `webp`, `gif`. Limit: 5 MB. Returns `{message, path}`.

15.9 Seeder

`seeder.js` connects via `config/db.js`, then with `--destroy` / `-d` only wipes the four collections, or without that flag runs:

1. Wipe Orders, Products, Categories, Users.

2. Insert users via `User.create()` so the bcrypt hook fires per doc.
3. `Category.insertMany(...)` and build a name → ObjectId map.
4. Map each product's `category` from the name in `data/products.js` to the resolved ObjectId.
5. `Product.insertMany(...)`.
6. For each product: pick 2–3 random non-admin users, fabricate review docs with rating 3–5 and a comment from a 14-string pool, recompute `rating` (avg) and `numReviews`, `await product.save()`.

Uses `colors` for friendly output. Exits 0 on success, 1 on any error.

15.10 Data files (used by the seeder)

- `data/users.js` — 1 admin + 4 customers with realistic French/Tunisian names. Plain-text passwords; the model hook hashes them.
 - `data/categories.js` — 6 entries (Electronics, Fashion, Home & Kitchen, Books, Sports & Outdoors, Beauty & Health).
 - `data/products.js` — 30 products (5 per category) with real product names, 2–3 sentence descriptions, realistic prices, stock 0–80, and verified Unsplash photo IDs in the format `https://images.unsplash.com/photo-<id>?w=600&q=80`.
-

16. Frontend — file-by-file reference

16.1 `main.jsx`

Wraps `<App />` in `<ErrorBoundary>` → `<Provider store={store}>` → `<BrowserRouter>` → `<ToastProvider>`. Imports global styles and the toast stylesheet. Strict mode enabled.

16.2 `App.jsx`

Renders `<Navbar>` + `<main><Routes>...</Routes></main>` + `<Footer>` inside `.app-shell`. Imports all stylesheets. Each `/profile`, `/checkout`, `/wishlist`, `/order/:id` is wrapped in `<PrivateRoute>`; `/admin/*` in `<AdminRoute>`. Fallback `*` route renders a 404 message.

16.3 `services/api.js`

Creates `axios.create({ baseURL: VITE_API_URL })`. Request interceptor reads `localStorage.userInfo`, parses it, and adds `Authorization: Bearer <token>` when available. Response interceptor passes errors through unchanged (slot reserved for future global 401 handling).

16.4 utils/imageUrl.js

- `imageUrl(path)` — if `path` is empty returns `/placeholder.svg`; if it starts with `http(s)` returns it unchanged; otherwise prefixes with the backend host (strips `/api` from `VITE_API_URL`). Lets the same field hold both an external CDN URL and a `/uploads/...` path.
- `onImageError(e)` — `onError` handler that swaps the `src` to the placeholder once (guards against infinite loops).
- `PLACEHOLDER` — exported constant so callers can configure a different fallback.

16.5 Redux

redux/store.js

`configureStore` with the five reducers (products, cart, user, order, wishlist). Adds a `createListenerMiddleware` that listens for any of the six cart actions (`addToCart` , `removeFromCart` , `updateQty` , `clearCart` , `saveShippingAddress` , `savePaymentMethod`) and writes `state.cart` to `localStorage.cart` after each change.

redux/slices/productSlice.js

Thunks:

- `fetchProducts({keyword?, category?, minPrice?, maxPrice?, sortBy?, page?, limit??})` — sends only present keys.
- `fetchProductById(id)`
- `createProduct(payload)` , `updateProduct({id, ...payload})` , `deleteProduct(id)` (admin)
- `createReview({productId, rating, comment})` — re-dispatches `fetchProductById` on success.
- `deleteReview({productId, reviewId})` — same refresh behaviour.

State: `{products, page, pages, total, product, loading, error}` . Reducers: `clearProduct` , `clearProductError` .

redux/slices/cartSlice.js

Pure reducers only (no thunks): `addToCart` (merges qty if same product), `removeFromCart` , `updateQty` , `clearCart` , `saveShippingAddress` , `savePaymentMethod` . Initial state rehydrated from `localStorage.cart` .

redux/slices/userSlice.js

Thunks: `login` , `register` , `logout` , `updateProfile` . Writes/removes `localStorage.userInfo` on success/logout. Initial `userInfo` rehydrated from storage.

redux/slices/orderSlice.js

Thunks: `createOrder`, `getOrderDetails`, `getMyOrders`, `payOrder` (manual), `listAllOrders` (admin), `deliverOrder` (admin). State `{order, orders, allOrders, loading, error, success}` + `resetOrderState`.

redux/slices/wishlistSlice.js

Thunks: `fetchWishlist`, `addToWishlist(product|id)`, `removeFromWishlist(id)`. Adds locally optimistically (filter for duplicates), removes by filtering by `_id`. Helper selector `selectIsWishlisted(productId)`.

16.6 Components

Navbar.jsx

- Logo → `/`.
- Search input with 300 ms debounce that navigates to `/search/<encoded>`. Submitting the form skips the debounce.
- Wishlist link with a numeric badge.
- Cart link with a quantity-sum badge.
- When `userInfo`: shows the name, the `Admin` badge if `isAdmin`, and a Logout button.
- On every `userInfo` change: dispatches `fetchWishlist()` (login) or `clearWishlist()` (logout).

Footer.jsx

Simple copyright line.

ProductCard.jsx

Image (with `onImageError`), heart overlay button (toggles wishlist; redirects to login when signed out), name, `Rating`, price and View link.

Rating.jsx

Renders 5 stars based on a numeric value (0–5). Uses `FaStar`, `FaStarHalfAlt`, `FaRegStar` from `react-icons`. Optional `text` prop for the label after the stars. `aria-label` for screen readers.

Loader.jsx

CSS-only spinner. Optional `size` prop.

PrivateRoute.jsx

If no `userInfo` → `<Navigate to="/login" replace>` (preserves the source location in `state.from`).

AdminRoute.jsx

If no `userInfo` → redirect to `/login`. If `userInfo` but not admin → redirect to `/`.

ErrorBoundary.jsx

Class component. Catches render errors, logs to the console, shows a friendly card with a "Try again" (reset) and "Go home" (location.assign) button. Collapsed `<details>` shows the technical error.

Toast.jsx

`ToastProvider` context + `useToast()` hook. API: `toast.success(msg)`, `toast.error(msg)`, `toast.info(msg)`, `toast.dismiss(id)`. Auto-dismiss after 3.5 s; manual close via X button. The stack is fixed top-right (full-width on mobile).

16.7 Pages

HomePage.jsx

Fetches latest products via `fetchProducts({})`. Renders a grid of `ProductCard`s plus a title.

SearchPage.jsx

Route param `:keyword?` plus URL query string for filters. Sidebar: category dropdown (loaded from `/api/categories`), price min/max, sort. Bottom: numbered pagination. Filter changes reset to page 1 and update `useSearchParams` so URLs are shareable.

ProductPage.jsx

Product details, qty selector capped by stock, "Add to Cart" + wishlist heart, then a Reviews section: existing reviews list (with trash icon for the owner/admin), then a form (rating dropdown + comment textarea) shown only when signed in and the user hasn't reviewed yet.

CartPage.jsx

Each line: image, name, price, qty `<select>`, remove button. Sidebar: items count + total. Checkout button redirects to `/login?redirect=/checkout` when signed out.

LoginPage.jsx / RegisterPage.jsx

Forms that dispatch `login` / `register`. On success → redirect (via `?redirect=` if present, else `/`).

ProfilePage.jsx

Update profile form on the left; user's orders table on the right. Each order has a link to `/order/:id`.

CheckoutPage.jsx

Shipping form, payment method radios (Stripe / PayPal / Cash on Delivery). On submit dispatches `createOrder`. When the thunk fulfils:

- **Stripe** → calls `POST /api/orders/:id/create-checkout-session`, then `window.location.href = data.url`.
- Else → navigates to `/order/:id` with a success toast.

WishlistPage.jsx

Grid of wishlisted products with "Add to Cart" + Remove on each card. Empty-state CTA when the list is empty.

OrderPage.jsx

Order details (shipping, payment, items list, total). Reads `?payment=success|cancelled` from the URL and shows the matching toast, then strips the param. Polls `getOrderDetails` every 2 s for up to 20 s after a Stripe success redirect, so the webhook has time to flip `isPaid`.

16.8 Admin pages

admin/DashboardPage.jsx

Four tile links to `/admin/products`, `/admin/categories`, `/admin/orders`, `/admin/users` with icons.

admin/ProductListAdmin.jsx

Table (thumb, name, price, category name, stock, edit/delete buttons). "New Product" button → `/admin/products/new`.

admin/ProductEditPage.jsx

Handles both `/admin/products/new` and `/admin/products/:id/edit`. Category dropdown is loaded from `/api/categories`. **Image input has a Upload-file / Use-URL toggle** — the URL mode is a free-text input, the upload mode posts to `/api/upload` (admin only). Preview image renders with `onError` fallback. Pre-fills the toggle to Upload mode when editing a product whose existing image is a `/uploads/...` path.

admin/CategoryListAdmin.jsx

Inline create form + per-row edit/save/cancel + delete. All API calls are direct via axios (no slice).

admin/OrderListAdmin.jsx

Table of all orders with paid/delivered badges. "Mark Delivered" button hides once the order is delivered.

admin/UserListAdmin.jsx

Table of users with an Admin/User badge. Toggle button switches `isAdmin`. Delete button. Both are disabled for the currently signed-in user.

16.9 Styles

Mobile-first. Variables in `global.css`: `--primary`, `--secondary`, `--bg`, `--text`, `--muted`, `--border`, `--danger`, `--success`, plus radius/shadow/font tokens.

- `global.css` — reset + tokens + typography.
- `toast.css` — toast stack + per-type accent + `ErrorBoundary` card.
- `navbar.css` — header layout, badge, search input, mobile collapsing.
- `product.css` — home grid, product card, product detail page, primary button.
- `cart.css` — cart list + summary; on mobile each row reflows to a `grid-template-areas` layout.
- `forms.css` — `.form-card` shared form styling, profile orders table.
- `admin.css` — dashboard tiles, table, icon buttons, badges, modal-style edit card, the Upload/URL toggle.
- `reviews.css` — star rating, heart buttons (card overlay + product page), review cards, review form.
- `search.css` — sidebar filters, sticky-on-desktop layout, pagination.
- `wishlist.css` — empty state + grid + card variant.
- `checkout.css` — payment-method hint, order-page two-column layout.

16.10 vite.config.js

`@vitejs/plugin-react` is registered. Dev server listens on port **5173** with `open: true` so the browser opens automatically.

16.11 public/placeholder.svg

Inline SVG with a gray rectangle, a mountain glyph, and the text "No image". Served by Vite at `/placeholder.svg`. Used by `onImageError`.

17. Library reference

This section explains every third-party library used, what it does and where you'll see it in the code.

17.1 Backend libraries

express

The HTTP server. Each `app.use(...)` registers middleware or a sub-router. The error middleware (`errorMiddleware.js`) is mounted last and matches any signature with four parameters (`err, req, res, next`).

mongoose

ODM for MongoDB. Used for schema definition, hooks (User pre-save), query builders (`Product.find(filter).populate('category').sort(sort).skip(...).limit(...)`), and atomic updates (`$addToSet`, `$pull` in the wishlist controller).

jsonwebtoken

`jwt.sign({id}, secret, {expiresIn: '30d'})` to create tokens, `jwt.verify(token, secret)` in `protect`.

bcryptjs

`bcrypt.hash(password, 10)` in the User pre-save hook, and `bcrypt.compare(plain, hashed)` in `matchPassword`.

dotenv

`dotenv.config()` populates `process.env` with the values from `.env` at process start.

cors

A permissive CORS middleware so the Vite dev server (port 5173) can call the API on port 5000.

multer

Multipart parser. We use `multer.diskStorage` with custom filename generation, a file filter to whitelist image MIME types, and a 5 MB limit. The route reads `req.file` after the middleware runs.

express-async-handler

Wraps an async controller in `Promise.resolve(fn(...)).catch(next)` so thrown errors propagate to the global error middleware without explicit try/catch.

stripe

Server-side Stripe SDK. Used for `stripe.checkout.sessions.create(...)` (Checkout Sessions in Payment mode) and `stripe.webhooks.constructEvent(rawBody, sigHeader, secret)` (signature verification).

colors

Adds string-prototype methods (`'hi'.green.bold`). Powers the friendly seeder output. (Modifying String.prototype is controversial but pragmatic; if you'd rather not, `chalk@4` is a drop-in alternative.)

nodemon (dev)

Watches files and restarts `node server.js` on change. Configured via the `dev` script.

17.2 Frontend libraries

react / react-dom

UI library. We use function components + hooks throughout. StrictMode in `main.jsx`.

react-router-dom (v6)

Used: `<BrowserRouter>` (in `main.jsx`), `<Routes>` / `<Route>` (in `App.jsx`), `<Link>` / `<Navigate>`, `useNavigate`, `useParams`, `useSearchParams`, `useLocation`.

@reduxjs/toolkit

`configureStore`, `createSlice`, `createAsyncThunk`, `createListenerMiddleware` (cart persistence in `store.js`), `isAnyOf` (matching multiple cart actions in the listener).

react-redux

`<Provider store={store}>`, `useSelector`, `useDispatch`.

axios

`axios.create({baseUrl})`. Request interceptor attaches the JWT from `localStorage`.

react-icons

Icon set from various icon libraries; we use Font Awesome (`fa`): `FaShoppingCart`, `FaUser`, `FaSearch`, `FaHeart`, `FaRegHeart`, `FaStar`, `FaStarHalfAlt`, `FaRegStar`, `FaTrash`, `FaEdit`, `FaPlus`, `FaArrowLeft`, `FaUserShield`, `FaBoxOpen`, `FaTags`, `FaClipboardList`, `FaUsers`, `FaCheck`, `FaCheckCircle`, `FaExclamationCircle`, `FaInfoCircle`, `FaTimes`.

@stripe/stripe-js

Loaded for future client-side use (Elements, Payment Element). The current implementation uses Stripe's hosted Checkout, so the redirect is a plain `window.location.href = url`; the library is available if you later want to embed Elements.

vite / @vitejs/plugin-react (dev)

Vite is the dev server and bundler. `npm run dev` for hot reload, `npm run build` for the production bundle to `dist/`.

18. Troubleshooting

Backend prints "Server running on port 5000" then exits

MongoDB connection failed. Common causes:

- Local Mongo service not running → `Start-Service MongoDB`.
- Atlas: IP not allowlisted → Atlas → Network Access → Allow `0.0.0.0/0` (development) or your current IP.
- Wrong password or special characters in the Atlas password → URL-encode them.
- DB name missing from the Atlas URI — make sure it's `...mongodb.net/<dbname>?...` .

Frontend "Operation `products.find()` buffering timed out after 10000ms"

Same root cause as above — the backend started but never connected to MongoDB. Check the backend terminal output for the `MongoDB Connected:` line.

```
npm error code ENOENT ... no such file or directory, open 'C:\...\package.json'
```

You're in the wrong folder. Most scripts live in `backend/` or `frontend/`. Either `cd` into the right folder or use `npm --prefix backend run <script>`.

```
The token '&&' is not a valid statement separator in this version.
```

PowerShell 5.1. Run one command per line, use `;`, or install PowerShell 7+.

Stripe webhook never marks an order paid

- Did you start `stripe listen --forward-to localhost:5000/api/orders/webhook`?
- Did you copy the printed `whsec_...` into `STRIPE_WEBHOOK_SECRET` and **restart** the backend?
- Is the `Stripe-Signature` header missing? Make sure the webhook route is mounted with `express.raw({ type: 'application/json' })` **before** `express.json()`. (It is — but if you re-organize `server.js`, preserve this order.)

Atlas hangs forever

Atlas blocks unknown IPs silently. Add `0.0.0.0/0` to Network Access.

"Already reviewed this product" when you've never reviewed it

Reviews are tracked per-user. If you switch accounts but a stale `userInfo` is in `localStorage`, sign out (clears storage) then sign back in.

Images not showing

- Confirm the URL pattern in the database. External Unsplash URLs start with `https://` — they bypass the host prefix. `/uploads/<file>` images need the backend running (Express serves them statically).
- `onImageError` swaps to `/placeholder.svg`; if you see that asset, the original URL failed.

19. Cheat sheet

```
# First-time setup
cd backend ; npm install ; Copy-Item .env.example .env
cd ..\frontend ; npm install

# Run
cd ..\backend ; npm run dev
# new terminal:
cd ..\frontend ; npm run dev

# Seed
cd ..\backend ; npm run seed
npm run seed:destroy

# Stripe
stripe login
stripe listen --forward-to localhost:5000/api/orders/webhook

# Useful URLs
# http://localhost:5173          Storefront
# http://localhost:5173/admin    Admin dashboard
# http://localhost:5000/api/products  API smoke test
# http://localhost:5000/uploads/<file>  Uploaded image
```

Credential	Email	Password
Admin	admin@shop.com	admin123
Customer	salma.bouid@gmail.com	password123
Customer	karim.trabelsi@outlook.com	password123
Customer	lucas.martin@gmail.com	password123
Customer	yasmine.mansouri@gmail.com	password123

Document generated for the MERN E-commerce project. See [README.md](#) in the repository root for the short-form version.